Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

SSRF RANLJIVOST IN ZAŠČITA

Raziskovanje spletnih ranljivosti in razvoj varnega spletnega okolja

Ivana Perić

Maribor, februar 2026

KAZALO

1 UVOD 1

2 KAJ JE SSRF? 2

3 PRIMER CAPITAL ONE NAPADA 4

4 OPIS PRAKTIČNEGA DELA 6

5 UPORABLJENE METODE IN TEHNOLOGIJE 8

6 NAVODILA ZA UPORABO 10

7 ZAKLJUČEK IN MOREBITNE IZBOLJŠAVE 12

8 VIRI 13

1 UVOD

Varnost spletnih aplikacij predstavlja enega ključnih izzivov sodobnega digitalnega okolja, saj so aplikacije vse bolj povezane z zunanjimi storitvami, oblačno infrastrukturo in notranjimi sistemi. Zaradi tega se povečuje tudi število varnostnih ranljivosti, ki lahko ob neustrezni obravnavi uporabniškega vnosa vodijo do resnih varnostnih incidentov. Ena izmed takšnih ranljivosti je **Server-Side Request Forgery (SSRF)**, ki omogoča napadalcu, da spletno aplikacijo prisili v izvajanje neželenih omrežnih zahtevkov.

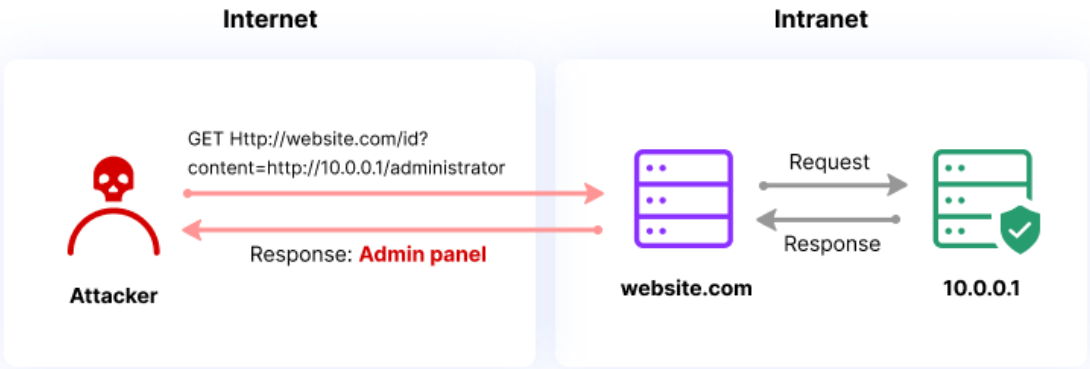
Namen tega projekta je raziskati delovanje SSRF ranljivosti ter prikazati njen vpliv na varnost spletnih aplikacij skozi praktični primer. Projekt združuje teoretično razlago ranljivosti z implementacijo dveh različic aplikacije – ranljive in varne – kar omogoča jasno primerjavo med neustrezno in pravilno zasnovano rešitvijo. Na ta način je mogoče bolje razumeti, kako SSRF napadi delujejo v praksi in zakaj predstavljajo resno tveganje, zlasti v okoljih oblačnih storitev.

Poseben poudarek projekta je na etičnem hekanju, saj je bila ranljivost analizirana in demonstrirana izključno v nadzorovanem testnem okolju. Praktični del temelji na simulaciji znanih napadalnih scenarijev, vključno z dostopom do notranjih metapodatkovnih storitev, brez uporabe dejanskih produkcijskih sistemov ali občutljivih podatkov.

Cilj projekta ni le prikazati, kako se SSRF ranljivost lahko izkoristi, temveč predvsem pokazati, kako jo je mogoče učinkovito preprečiti z ustreznimi metodami varnega programiranja. Projekt tako prispeva k boljšemu razumevanju pomena varnosti v razvoju spletnih aplikacij ter poudarja odgovorno in etično ravnanje pri raziskovanju varnostnih ranljivosti.

2 KAJ JE SSRF?

Server-Side Request Forgery (SSRF) je varnostna ranljivost spletnih aplikacij, pri kateri napadalec izkoristi funkcijo, ki dovoljuje strežniku, da sam pošlje HTTP zahteve na določene URL-je. Ranljivost nastane, kadar aplikacija sprejme ali uporabi uporabniški vhod (npr. URL), ki ga nato strežnik uporabi za mrežni poziv, ne da bi preveril, ali je ta URL varen ali dovoljen. Zaradi tega lahko napadalec prisili strežnik, da izvede klice na notranje ali druge nepričakovane vire, ki običajno niso izpostavljeni javnemu internetu. (Semgrep, b.d)



Slika 2.1: Kako SSRF dela

(Vir: Imperva, b.d)

Ključna nevarnost SSRF izvira iz tega, da **zahtevki izvirajo iz strežnika same aplikacije**, ki ima običajno več pravic in dostopa do notranjih omrežij ali zaščitenih storitev. To pomeni, da napadalec lahko izkoristi to ranljivost za dostop do sistemov, do katerih sam ne bi mogel dostopati neposredno. (Semgrep, b.d)

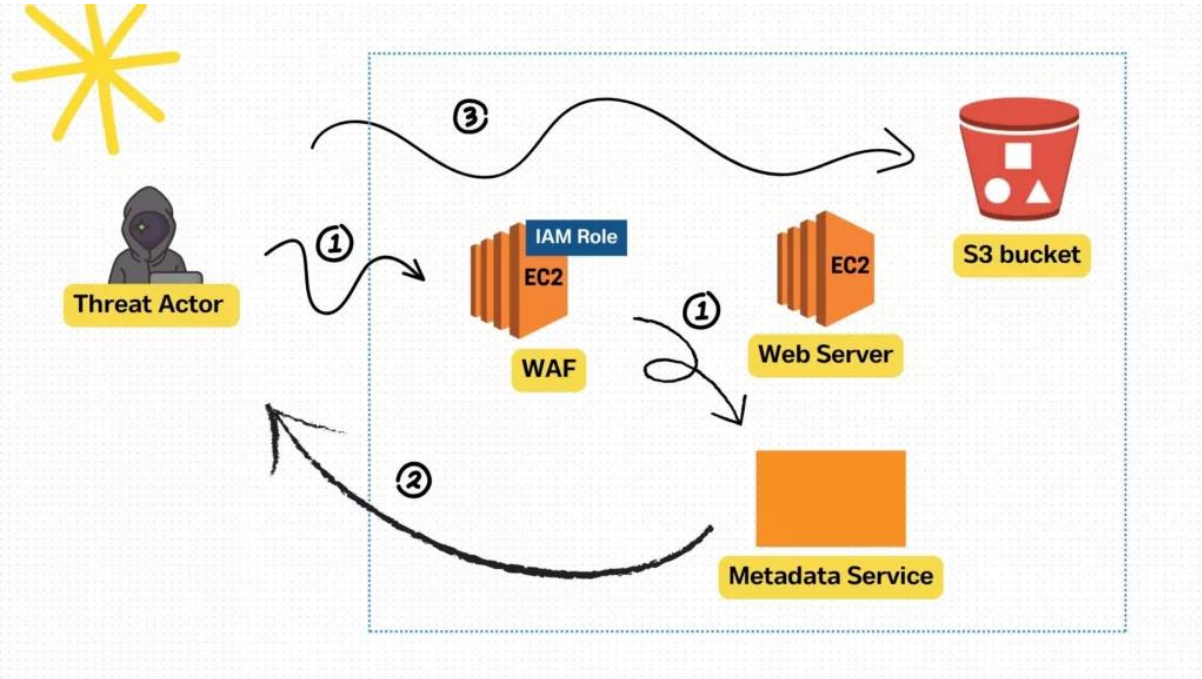
V praksi se SSRF pogosto pojavi v scenarijih, kjer aplikacija omogoča nalaganje vsebin ali podatkov z zunanjih URL-jev, na primer pri funkcijah za prikaz slik, uvoz datotek ali klic API-jev na podlagi URL-jev, ki jih vnese uporabnik. Če ti URL-ji niso ustrezno preverjeni ali omejeni, lahko napadalec zamenja legitimne URL naslove z zlonamernimi ali notranjimi naslovi ter tako prisili strežnik, da izvede neželene klice. (Mozilla Contributors, 2025)

SSRF je zaradi svojega potencialnega vpliva na notranje vire in cloud infrastrukturo uvrščena med resne varnostne ranljivosti in je tudi del **OWASP API Security Top 10**, kar poudarja njen pomen v sodobnih aplikacijah, še posebej tistih, ki delujejo v razširjenih omrežjih ali oblačnih okoljih. (OWASP API Security Project, 2023)

3 PRIMER CAPITAL ONE NAPADA

V letu **2019 je finančna institucija *Capital One*** doživela enega največjih *cloud* podatkovnih vdorov v zgodovini, ki je bil povezan z ranljivostjo **Server-Side Request Forgery (SSRF)**. V tem incidentu je napadalec izkoristil konfiguracijsko ranljivost v **Web Application Firewall (WAF)**, ki je tekel na infrastrukturi AWS, kar mu je omogočilo pošiljanje neželenih zahtevkov na notranji **AWS Instance Metadata Service**. To je povzročilo, da je strežnik razkril začasne AWS poverilnice (*credentials*), ki so jih napadalcu omogočile dostop do **S3 bucketov** z občutljivimi podatki strank. S pomočjo teh poverilnic je napadalec pridobil in izvozil osebne podatke več kot **100 milijonov posameznikov** iz ZDA in Kanade, vključno z imeni, naslovi, datumi rojstva, številkami socialnih zavarovanj, bančnimi podatki in drugimi informacijami (Ionix, 2025; vectra.ai, 2024).

Incident je bil odkrit 19. julija 2019, ko je *Capital One* izvedel, da je nepooblaščen posameznik pridobil dostop do občutljivih podatkov. Do vdora je prišlo med marcem in julijem istega leta, preden je bila ranljivost odkrita in odpravljena. FBI je pozneje aretiral napadalca, *Paige Thompson*, nekdanjo zaposleno v podjetju *Amazon Web Services*, ki je izkoristila SSRF ranljivost za dostop do AWS infrastrukture *Capital Onea* (Capital One, 2019).



Slika 3.1: Diagram napada na CapitalOne

(Vir: InsiderSecurity, 2025)

Ključni element vdora je bila ranljivost, ki je dovoljevala, da je WAF sprejel manipulirane zahteve in nato poslal notranje HTTP klice na **AWS EC2 metadata endpoint** (http://169.254.169.254/latest/meta-data/). Ker AWS metadata storitev omogoča strežnikom, da pridobijo informacije in *temporary credentials* brez dodatne avtentikacije, je imel napadalec s tem dostop do AWS poverilnic in posledično do notranjih virov, kot so S3 bucketi z osebnimi podatki. Ta tehnika izkoriščanja SSRF ranljivosti je ključna lekcija v zavedanju, kako nevarne so lahko konfiguracijske napake v *cloud* okoljih (Fastly, 2019; vectra.ai, 2024).

Incident Capital One jasno kaže, da tudi konfiguracijske ranljivosti, ki se zdijo na prvi pogled majhne, lahko privedejo do katastrofalnih posledic, če jih napadalci kombinirajo z dostopom do notranjih oblačnih storitev in API-jev. Ranljivost je poudarila potrebo po bolj sofisticiranem pristopu k varnosti spletnih aplikacij in *cloud* infrastrukture, še posebej v kontekstu pravilnega upravljanja z IAM vlogami in nadzora dostopa do občutljivih virov.

4 OPIS PRAKTIČNEGA DELA

Praktični del projekta je sestavljen iz dveh ločenih spletnih strežnikov, ki jasno ponazarjata razliko med **ranljivo implementacijo SSRF** in **varno implementacijo z zaščitnimi mehanizmi**. Cilj praktičnega dela je prikazati, kako lahko napačna obdelava uporabniškega vnosa vodi v SSRF ranljivost ter kako se ta ranljivost lahko učinkovito prepreči z večslojno validacijo in nadzorom omrežnih zahtevkov.

4.1 Ranljiva aplikacija

Ranljiva različica aplikacije je implementirana v datoteki server-vulnerable.js. Aplikacija temelji na okolju **Node.js** in ogrodju **Express** ter vsebuje endpoint /fetch, ki kot parameter sprejme URL in ga brez kakršnekoli validacije uporabi za izvedbo HTTP zahtevka na strežniški strani.

Takšna implementacija predstavlja klasičen primer SSRF ranljivosti, saj aplikacija neposredno zaupa uporabniškemu vnosu in omogoča, da strežnik pošilja zahteve na poljubne cilje. Napadalec lahko ta endpoint zlorabi tako, da kot URL posreduje naslov notranjega vira. V okviru projekta je za demonstracijo vzpostavljen tudi simuliran **AWS metadata endpoint** (/aws-metadata/latest/meta-data/iam/security-credentials), ki vrača lažne poverilnice. S tem je prikazano, kako lahko SSRF napad vodi do razkritja občutljivih podatkov, podobno kot v realnem primeru napada na Capital One.

V ranljivi aplikaciji ni implementiranih nobenih omejitev glede dovoljenih protokolov, domen ali IP naslovov, prav tako ni zaščite pred preusmeritvami ali časovnimi zlorabami zahtevkov. To jasno pokaže, kako nevarna je lahko takšna implementacija v produkcijskem okolju.

4.2 Varna aplikacija

Varna različica aplikacije je implementirana v datoteki server-secure.js in uporablja centralizirano varnostno logiko, definirano v modulu secure_route.js. Ta pristop omogoča boljšo preglednost, ponovljivo uporabo varnostnih pravil in zmanjšanje možnosti napak pri implementaciji.

Ključni del varne implementacije je funkcija validateAndFetch, ki pred izvedbo HTTP zahtevka izvede več zaporednih varnostnih preverjanj. Najprej se preveri veljavnost URL-ja in omeji uporaba protokolov izključno na http in https. Nato se preveri, ali je domena vključena v **allowlist**, kar pomeni, da aplikacija lahko dostopa le do vnaprej določenih in zaupanja vrednih gostiteljev.

Poleg tega aplikacija blokira znane nevarne cilje, kot so lokalni naslovi, cloud metadata naslovi in specifične poti, ki so značilne za dostop do metapodatkov. Z uporabo DNS poizvedbe se hostname razreši v IP naslov, nato pa se preveri, ali gre za zasebni ali *link-local* naslov. Če je zaznan tak naslov, se zahtevek zavrne.

Dodatno varnost predstavlja onemogočeno samodejno sledenje HTTP preusmeritvam ter časovna omejitev zahtevkov, ki preprečuje zlorabe v obliki počasnih ali neskončnih povezav. Vsi poskusi dostopa, tako dovoljeni kot blokirani, se beležijo s pomočjo mehanizma za logiranje, kar omogoča analizo in zaznavanje sumljivih vzorcev.

4.3 Primerjava ranljive in varne implementacije

Primerjava obeh pristopov jasno pokaže razliko med neustrezno in pravilno obravnavo uporabniškega vnosa. Medtem ko ranljiva aplikacija omogoča neposredno izvajanje SSRF napada, varna aplikacija z večslojno zaščito uspešno prepreči dostop do notranjih virov, tudi če napadalec poskuša uporabiti znane tehnike, kot so manipulacija URL-jev, dostop do metadata storitev ali izkoriščanje preusmeritev.

Praktični del projekta tako ponuja realističen in uporaben primer, ki prikazuje, kako je mogoče SSRF ranljivost reproducirati v nadzorovanem okolju ter kako jo je mogoče učinkovito odpraviti z ustreznimi varnostnimi mehanizmi.

5 UPORABLJENE METODE IN TEHNOLOGIJE

Pri razvoju praktičnega dela projekta so bile uporabljene sodobne spletne tehnologije in varnostni pristopi, ki omogočajo realistično simulacijo SSRF ranljivosti ter implementacijo ustreznih zaščitnih mehanizmov. Izbrane metode in tehnologije so primerne za demonstracijo problema v nadzorovanem okolju in hkrati odražajo prakse, ki se uporabljajo v realnih spletnih aplikacijah.

5.1 Tehnološki sklad

Projekt je implementiran v okolju **Node.js**, ki omogoča asinhrono obdelavo HTTP zahtevkov in je pogosto uporabljen pri razvoju sodobnih spletnih API-jev. Kot ogrodje za razvoj strežniškega dela je uporabljen **Express**, ki omogoča enostavno definicijo REST endpointov in obdelavo uporabniških zahtevkov.

Za izvajanje HTTP zahtevkov na strežniški strani je uporabljena standardna funkcija fetch, kar omogoča jasno demonstracijo, kako lahko neustrezna uporaba takšnih funkcij vodi v SSRF ranljivost. Aplikacija je razdeljena na več modulov, kar izboljša berljivost kode in omogoča ločeno obravnavo varnostne logike.

5.2 Metode demonstracije ranljivosti

Za prikaz SSRF ranljivosti je bila uporabljena metoda **namerne implementacije ranljive funkcionalnosti**, kjer aplikacija sprejme URL kot uporabniški parameter in ga neposredno uporabi za izvedbo HTTP zahtevka. Ta pristop omogoča jasno razumevanje, kako SSRF napad deluje v praksi in zakaj predstavlja resno varnostno tveganje.

Poleg tega je bil vzpostavljen **simuliran notranji vir**, ki posnema delovanje AWS Instance Metadata Service. Ta simulacija omogoča demonstracijo posledic SSRF napada brez uporabe dejanske oblačne infrastrukture, kar je skladno z etičnimi načeli predmeta in preprečuje morebitne zlorabe.

5.3 Varnostni pristopi in zaščitni mehanizmi

V varni različici aplikacije je uporabljen večslojni varnostni pristop, ki temelji na kombinaciji naslednjih metod:

- **Validacija URL-jev**, kjer se preveri sintaktična pravilnost in dovoljen protokol (samo HTTP in HTTPS).
- **Allowlist pristop**, ki omeji dostop zgolj na vnaprej določene in zaupanja vredne domene.
- **Blokiranje znanih nevarnih gostiteljev in poti**, kot so lokalni naslovi in metadata endpointi.
- **DNS razreševanje in preverjanje IP naslovov**, s čimer se zazna in prepreči dostop do zasebnih ali *link-local* omrežij.
- **Nadzor nad HTTP preusmeritvami**, kjer je samodejno sledenje preusmeritvam onemogočeno, saj lahko te vodijo do obhoda varnostnih pravil.
- **Časovne omejitve zahtevkov**, ki zmanjšujejo tveganje zlorab in zagotavljajo stabilnost aplikacije.
- **Logiranje poskusov dostopa**, kar omogoča kasnejšo analizo in zaznavanje sumljivih dejavnosti.

Ta kombinacija metod predstavlja praktičen primer obrambnega programiranja (*defensive programming*) in sledi priporočilom za preprečevanje SSRF ranljivosti v sodobnih spletnih aplikacijah.

5.4 Metodološki pristop

Projekt je bil razvit po principu **primerjalne analize**, kjer se ranljiva in varna implementacija obravnavata vzporedno. Tak pristop omogoča boljše razumevanje varnostnih tveganj in jasno pokaže, kako posamezni zaščitni mehanizmi vplivajo na preprečevanje napadov. Metodologija temelji na etičnem hekanju, saj je bil napad simuliran izključno v testnem okolju in z namenom izboljšanja varnosti.

6 NAVODILA ZA UPORABO

Za preizkus praktičnega dela projekta je potrebno lokalno zagnati obe različici strežnika – ranljivo in varno – ter nato primerjati njuno obnašanje pri enakih zahtevkih.

6.1 Namestitev in zagon

Najprej je potrebno namestiti vse odvisnosti projekta. V terminalu se premaknemo v mapo backend in zaženemo namestitev:

npm install

Po uspešni namestitvi je treba zagnati oba strežnika v ločenih terminalih.

Zagon ranljivega strežnika:

node server-vulnerable.js

Ranljivi strežnik teče na portu **3000**.

Zagon varnega strežnika:

node server-secure.js

Varni strežnik teče na portu **4000**.

Po zagonu oba strežnika izpišeta statusno sporočilo, ki potrjuje, da aplikacija uspešno deluje.

6.2 Preizkus ranljive različice

Ranljiva različica omogoča neposredno pošiljanje URL-jev brez kakršnekoli validacije. To lahko preizkusimo z orodjem curl ali z brskalnikom.

Primer SSRF napada na ranljivi strežnik:

curl "http://localhost:3000/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials"

Strežnik vrne simulirane AWS poverilnice, kar ponazarja uspešen SSRF napad in razkritje notranjih podatkov. Ta primer prikazuje, kako nevarna je uporaba uporabniškega vnosa brez ustreznih varnostnih preverjanj.

6.3 Preizkus varne različice

Na varnem strežniku se enak zahtevek zavrne zaradi vgrajenih zaščitnih mehanizmov.

Primer istega napada na varni strežnik:

curl "http://localhost:4000/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials"

Strežnik vrne sporočilo o napaki, saj cilj ni na seznamu dovoljenih gostiteljev. S tem je prikazano, da varna implementacija uspešno prepreči SSRF napad.

Legitimne zahteve na dovoljene zunanje vire še vedno delujejo pravilno, na primer:

curl "http://localhost:4000/fetch?url=http://httpbin.org/get"

6.4 Samodejno testiranje

Projekt vsebuje tudi skripto za samodejno testiranje, ki zaporedno preveri različne scenarije napadov in legitimnih zahtevkov.

Skripto zaženemo z ukazoma:

chmod +x scripts/ssrf-tests.sh

./scripts/ssrf-tests.sh

Skripta prikaže:

- uspešno izkoriščanje SSRF na ranljivi aplikaciji,
- blokiranje enakega napada na varni aplikaciji,
- pravilno delovanje legitimnih zahtevkov,
- blokado dostopa do lokalnih in zasebnih IP naslovov.

7 ZAKLJUČEK IN MOREBITNE IZBOLJŠAVE

V okviru projekta je bila analizirana in praktično predstavljena varnostna ranljivost **Server-Side Request Forgery (SSRF)**, ki se pogosto pojavlja v sodobnih spletnih aplikacijah, zlasti v okoljih oblačnih storitev. S pomočjo primerjave ranljive in varne implementacije je bilo prikazano, kako lahko napačna obravnava uporabniškega vnosa omogoči napadalcu dostop do notranjih virov ter kako je mogoče takšne napade učinkovito preprečiti z ustreznimi zaščitnimi mehanizmi.

Praktični del projekta jasno pokaže, da odprava SSRF ranljivosti ne zahteva nujno kompleksnih ali dragih rešitev, temveč predvsem dosledno uporabo načel varnega programiranja. Uporaba validacije URL-jev, omejevanje dovoljenih ciljev, preverjanje DNS zapisov in blokiranje zasebnih IP naslovov predstavlja učinkovit večslojni obrambni pristop, ki bistveno zmanjša napadalno površino aplikacije.

Kljub temu projekt ponuja več možnosti za nadaljnje izboljšave, ki bi rešitev še približale realnim produkcijskim okoljem. Ena izmed ključnih nadgradenj bi bila uvedba **omejevanja hitrosti zahtevkov (rate limiting)**, s čimer bi preprečili avtomatizirane poizkuse napadov in zmanjšali možnost zlorab. Prav tako bi bilo smiselno razširiti **logiranje in nadzor**, na primer z dodajanjem različnih stopenj resnosti dogodkov ter integracijo z nadzornimi sistemi za zaznavanje varnostnih incidentov.

Dodatno bi bilo mogoče izboljšati upravljanje varnostnih pravil z ločitvijo konfiguracije od aplikacijske kode, kar bi omogočilo bolj prilagodljivo vzdrževanje in hitrejše odzivanje na nove grožnje. Prav tako bi bilo smiselno razmisliti o naprednejši zaščiti pred DNS-based napadi ter bolj granularni obravnavi HTTP preusmeritev, namesto njihove popolne blokade.

Projekt bi lahko v prihodnje razširili tudi z integracijo na infrastrukturnem nivoju, na primer z uporabo pravil Web Application Firewall ali omrežnih omejitev, kar bi dodatno okrepilo varnost aplikacije. Takšen celosten pristop bi omogočil boljšo zaščito pred SSRF napadi in hkrati prikazal, kako se varnost izvaja na več nivojih sistema.

8 VIRI

Capital One. (2019). *2019 Capital One cyber incident: What happened*. https://www.capitalone.com/digital/facts2019/

Fastly. (2019). *Preventing server-side request forgery (SSRF)*. https://www.fastly.com/blog/preventing-server-side-request-forgery-ssrf

Imperva. (b.d.). *Server-side request forgery (SSRF)*. Imperva Learning Center. https://www.imperva.com/learn/application-security/server-side-request-forgery-ssrf/

InsiderSecurity. (2025, January 6). *CapitalOne data breach: How SSRF vulnerability exposed 100 million customer records*. https://insidersecurity.co/capitalone-data-breach-how-ssrf-vulnerability-exposed-100-million-customer-records/

IONIX. (2025). *OWASP Top 10: Server-side request forgery (SSRF) case study: Capital One*. https://www.ionix.io/guides/owasp-top-10/server-side-request-forgery-ssrf/

Mozilla Contributors. (2025, December 5). *Server Side Request Forgery (SSRF)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/SSRF

OWASP API Security Project. (2023). *API7:2023 Server Side Request Forgery (SSRF)*. OWASP. https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/

Semgrep. (b.d.). *Server-side request forgery*. Semgrep Documentation. https://semgrep.dev/docs/learn/vulnerabilities/server-side-request-forgery

Vectra AI. (2024). *Metadata: The hidden data powering cyber defense and the Capital One breach*. https://www.vectra.ai/topics/metadata